单卡推理服务：trace 回放基线与低精度对照

本文报告单卡推理服务的两组真实测量：用公开 trace 开环回放得到的 SLO goodput 基线，以及同一模型 BF16 与在线 FP8 的吞吐、延迟和留出集精度对照。另有一组投机解码测量，目标模型换成了更小的模型。计划里的 P/D 分离、跨节点和更大的 MoE 模型都还没有做（原因见“局限”）。[1]

方法

回放器与有效性规则

Go 写的开环回放器按 trace 的到达时间发请求，记录计划发送时间和实际发送时间。一轮里如果发送滞后超过 5 ms 的请求占比超过 1%，这一轮作废；超时和失败都留在分母里。[2] [3] prompt 是按 trace 长度生成的合成 token，输出长度用 `ignore_eos` 和 `min_tokens` 强制成 trace 的值。[2]

基线

服务端是 vLLM 0.10.2，模型 Qwen3-8B-AWQ（4-bit，vLLM 按 `awq_marlin` 运行），KV 缓存 11.90 GiB = 86,624 token。[2] 负载用 Azure LLM inference 2023 conv（校准 191 个、评测 265 个请求）和 BurstGPT_1（105 / 123 个请求）。[2] Azure 的 1x 定为源速率的 0.25×，所以 4x 就是 trace 的真实速率；BurstGPT 的 1x 就是源速率。[2]

SLO 由三次 1x 校准运行得出：p99 TTFT 和 p99 TPOT。goodput 是同时满足两项的请求数除以从第一次发送到最后一次完成的时间。[2] 每档各跑 3 轮，轮次顺序轮换，报告中位数和 [最小, 最大]。[3]

低精度对照

同一个 Qwen3-8B checkpoint：基线用 BF16 权重，候选加 vLLM 在线 FP8（`--quantization fp8`）。[4] 负载是 Azure 评测窗口的 0.25× 源速率，运行顺序 AB、BA、AB 共 6 次，每次换精度时重启服务。[4] SLO 来自 3 次 BF16 校准：TTFT ⩽ 1500.4 ms，TPOT ⩽ 40.08 ms。[4] 精度在 200 道 GSM8K 留出题和 300 道 MMLU-Pro 题上用贪心解码测，同一 checkpoint 的两个精度用完全相同的 prompt。[4]

结果

基线 goodput

SLO：Azure TTFT ⩽ 460.7 ms、TPOT ⩽ 21.51 ms；BurstGPT TTFT ⩽ 203.5 ms、TPOT ⩽ 11.37 ms。[3]

Trace	档位	goodput（req/s）	达标 / 发送	超时	p50 / p99 TPOT（ms）	证据 [3]
Azure	1x	1.089 [1.089, 1.089]	263/265	0	9.5 / 16.1	[3]
Azure	2x	2.041 [2.040, 2.041]	252/265	0	12.3 / 23.1	[3]
Azure	4x	0.104 [0.104, 0.104]	7/265	0	33.2 / 60.6	[3]
BurstGPT	1x	0.401 [0.401, 0.405]	119–120/123	0	8.0 / 9.0	[3]

Trace	档位	goodput（req/s）	达标 / 发送	超时	p50 / p99 TPOT（ms）	证据 [3]
BurstGPT	2x	0.812 [0.805, 0.812]	120–121/123	0	8.0 / 9.9	[3]
BurstGPT	4x	1.502 [1.502, 1.529]	113–115/123	0	9.1 / 14.0	[3]

Azure 在 4x 时饱和：输出吞吐到 1,141 tok/s，p50 TPOT 升到 33 ms（SLO 21.5 ms），265 个请求里 258 个虽然完成但不达标。[3] 拐点在 Azure 源速率的 0.5× 到 1.0× 之间。[3] 评测里没有失败；每次 BurstGPT 校准有一个 HTTP 400，来自源数据里输入为 0 token 的一行，它留在计数里。[3] 每轮发送滞后的请求最多 1/191 = 0.5%，低于作废线。[3]

精度对照

所有 6 次运行都 sender-valid，每次 265/265 成功、0 超时。[1]

指标	BF16	FP8	差	证据 [1]
输出吞吐（tok/s）	311.02 [311.00, 311.04]	315.35 [315.33, 315.37]	+1.4%	[1]
SLO goodput（req/s）	1.065 [1.065, 1.065]	1.088 [1.088, 1.088]	+2.2%	[1]
p50 / p99 TTFT（ms）	128 / 485	89 / 329	−30% / −32%	[1]
p50 / p99 TPOT（ms）	24.6 / 31.8	15.5 / 20.9	−37% / −34%	[1]
KV 容量（token）	22,192	68,016	×3.1	[1]
GSM8K 准确率（200）	0.920	0.910	−0.010	[1]
MMLU-Pro 准确率（300）	0.377	0.380	+0.003	[1]

精度的一个标准误约为 GSM8K ±1.9 pp、MMLU-Pro ±2.8 pp，两个精度差都不能和零区分。[1] 在这个负载下吞吐由到达速率决定，BF16 每次也只有 2 个请求不达标，所以 FP8 的收益体现在延迟和 KV 容量上，没有体现在 goodput 上。[1]

第一次精度评测失败后保留为证据，没有使用：MMLU-Pro 的 prompt 没有限制输出，300 条 BF16 回答里有 256 条被截断、无法解析。[5] 修正后两个精度用同样的 prompt 重跑，没有按结果调 prompt。

投机解码

用 SpecForge（固定 `e3ab28d`）给 Qwen3-4B 训练了 EAGLE-3 draft，再在 SGLang 0.5.18 上和不开投机解码的同一模型对照。[6] 两种服务交替各跑 2 轮，每轮用同样的 128 个留出 prompt，贪心解码，最多 256 个新 token。[6]

batch	基线 TPOT（ms）	EAGLE-3 TPOT（ms）	接受长度	证据 [7]
1	9.34 / 9.34	8.28 / 8.26	1.423 / 1.421	[7]
4	10.20 / 10.18	8.89 / 8.73	1.420 / 1.427	[7]
16	11.05 / 11.05	10.37 / 10.34	1.419 / 1.424	[7]
64	13.55 / 13.84	17.56 / 17.25	1.421 / 1.424	[7]

batch 1 到 16 的 TPOT 降低 6–14%；batch 64 时目标模型已经不受显存带宽限制，每步验证 4 个 token 的代价超过了约 1.42 个接受 token 的收益，TPOT 反而升高 25–30%。[7] 每种模式 1,024 个请求全部完

成，0 个失败。[7]

第七阶段换成 SpecForge 标准大小的 draft（vocab 32000、intermediate 9728），用 60,697 段由 Qwen3-4B 重新生成的 UltraChat 对话训练。训练离线分 105 批、每批 576 段，共 60,480 步。[8] 测量方法和上表相同，同样的 128 个留出 prompt，各 2 轮，0 个失败。

batch	基线 TPOT（ms）	EAGLE-3 TPOT（ms）	TPOT 加速	接受长度	证据 [8]
1	9.31 / 9.32	5.66 / 5.65	1.646 / 1.649	2.159 / 2.159	[8]
4	10.15 / 10.16	6.07 / 6.07	1.670 / 1.673	2.152 / 2.152	[8]
16	11.02 / 11.03	7.26 / 7.22	1.518 / 1.528	2.156 / 2.160	[8]
64	13.62 / 13.45	11.78 / 11.87	1.157 / 1.133	2.165 / 2.164	[8]

接受长度从 1.42 升到 2.16，batch 64 也变成 TPOT 降低 12–14%、吞吐提高 6%。[8] 公开的 AngelSlim/Qwen3-4B_eagle3（Apache-2.0）接受长度 2.11，在这些 UltraChat prompt 上和我们的 draft 相差 1–3%。[8] 事先声明的配置网格只测 batch 1 和 64：topk 4、8 个 draft token 在 batch 1 最快（1.88–1.90×），2 步链在 batch 64 最快（1.29×）。[8] 共用了约 9.3 GPU 小时，机器上同时有其他会话在跑。[8]

局限

- 每轮负载是确定的（相同到达、固定 prompt、强制长度），所以 3 轮几乎相同，范围只反映运行噪声，不反映不同 trace 样本之间的差异。[3]
- p99 SLO 来自几百个校准样本，接近样本最大值。[9]
- 为了适配一张卡，用 vLLM 代替 SGLang，用 Qwen3-8B AWQ 代替原计划的 MoE 模型；FP8 对照两边都用 eager 模式，绝对延迟比 CUDA graph 高。GPU 和另一个项目共用。[9] [10]
- 在线 FP8 用每张量动态缩放，不是校准过的 FP8 checkpoint。[10]
- P/D 分离、跨节点、KV 走 RDMA 和超大模型都暂停，等多机 GPU 预算：未在真实多机上验证。P/D 布局只是 `serving/deploy/pd-*.yaml` 里不启用的离线工件。
- 路由器与扩缩容在单卡上的真实运行：4× 突发下静态单副本的 goodput 是 4.00 req/s，扩缩容只有 0.675 req/s，因为两个副本进程共用一张卡，p50 TPOT 从 3.7 涨到 5.6 ms；所以单卡结果不能说明扩容的收益。[11]
- 杀掉一个副本时，3 次运行里每次 260 个请求都落到成功或失败，没有丢失；每次 32–34 个失败，集中在路由器快照过期之前的约 15 s 内。[12]
- 之后路由器改为遇到传输错误立即摘除后端：同样的杀进程实验里，每次失败降到 2 / 2 / 3 个；加上连接失败时重发一次（`-retry-dial`），降到 1 / 0 / 1 个，剩下的只有杀进程时正在流式输出的请求。摘除发生在 kill 后 0.44 到 1.25 s。[13]
- 第六阶段的投机解码受磁盘份额限制：draft 只训练了 240 段对话（约 0.15M token），并把 draft 缩小了，所以接受长度只有约 1.42。第七阶段的正式规模 draft 解决了这一点，但只用了一个数据集（UltraChat），留出 prompt 也来自 UltraChat，可能偏向我们的 draft。另外，原计划的目标模型 Qwen3-30B-A3B 放不进一张 4090。[6] [14]

复现

运行命令、提交、模型修订、环境哈希和 seed 见复现附录。原始输出在 `docs/evidence/serving-baseline/` 和 `docs/evidence/fp8-vs-bf16/`，包括每次运行的回放 JSON、服务日志、`summary.json`，以及 `serving/eval/accuracy.py` 生成的报告（`comparison_eligible: true`）。[10]

证据引用

1. `docs/evidence/pd-disaggregation.md#results`
2. `docs/evidence/serving-baseline.md#setup`
3. `docs/evidence/serving-baseline.md#results`
4. `docs/evidence/pd-disaggregation.md#setup`
5. `docs/evidence/pd-disaggregation.md#quality-attempt-1-kept-not-used`
6. `docs/evidence/spec-decoding.md#setup`
7. `docs/evidence/spec-decoding.md#results`
8. `docs/evidence/spec-decoding.md#phase-7-2026-11-01-full-size-draft-60697-regenerated-conversations`
9. `docs/evidence/serving-baseline.md#limits`
10. `docs/evidence/pd-disaggregation.md#limits-and-paused-parts`
11. `docs/evidence/autoscaling.md#task-14-local-router--autoscaler-on-one-rtx-4090--2026-10-27`
12. `docs/evidence/serving-faults.md#results`
13. `docs/evidence/serving-faults.md#update-phase-4-router-ejects-on-transport-errors--loss-3234--03-per-kill`
14. `docs/evidence/spec-decoding.md#limits`